

InferQ: A Database-Oriented Benchmark for Quantum Circuits Simulation: [Experiments & Analysis]

Abstract

Recent work suggests that relational database management systems (RDBMSs) can execute quantum circuit simulation by compiling the simulation into SQL workloads (primarily join-and-aggregate tensor contractions). While early results are promising, they largely focus on a narrow set of highly structured circuits and offer limited support for systematic database research, such as query optimization, physical design, and engine-level evaluation across a broad range of circuits. We present InferQ, a database-oriented benchmark for quantum circuit simulation. InferQ generates *general*, *compositional* circuits by assembling subcircuits from a set of circuit templates, emits each simulation task as an RDBMS-ready SQL workload, and extracts circuit and query features (static, graph, SQL, and dynamic) for workload characterization. InferQ also releases a large dataset of 202,975 circuits online, with a web-based viewer to support searching, filtering, and downloading circuits and feature records. In experiments across three RDBMS engines (PostgreSQL, SQLite, DuckDB) and the widely used Qiskit Aer simulator, we find that even without database-specific tuning, SQLite achieves better peak memory usage than Qiskit Aer on more than 40% of the circuits generated by InferQ. Moreover, using InferQ features, lightweight machine learning models (linear and tree-based models) can accurately predict when SQL execution is preferable (with accuracy up to 95.3% for runtime and 97.6% for memory), enabling data-centric simulator selection and opening the door to principled optimization of SQL-based quantum circuit simulation.

1 Introduction

In the current noisy intermediate-scale quantum (NISQ) era, most quantum applications are developed and validated through classical simulation. Quantum programs are typically expressed in the circuit model as a sequence of gates, and the common practice is to execute the circuit on a simulator before running it on quantum hardware. Simulators produce the circuit’s (predicted) output, e.g., measurement outcomes or their probabilities. Efficient circuit simulation, therefore, remains a cornerstone for near-term quantum algorithm development.

Data management challenge. Recent work [5, 9] argues that relational database management systems (RDBMSs) can serve as a competitive simulation *engine* by compiling circuit simulation into relational workloads. A particularly promising result is that DuckDB can scale to *millions of qubits* for sparse circuits. Concretely, under a 2GB memory limit, DuckDB simulates up to 4,000,862 qubits for GHZ-state preparation and up to 53,017 qubits for W-state preparation [9], which are two standard state-preparation routines that initialize many qubits into widely used multi-qubit entangled states. These results raise an obvious question for the database community: *when does “an RDBMS as a simulator” actually work, and when does it break down?*

The same study makes clear this scalability is highly workload-dependent: for a dense circuit such as Quantum Fourier Transform (QFT), the maximum supported qubit count is only 14 under the same setting. Moreover, the largest improvements in those experiments arise from circuits with a specific structure. For instance, GHZ is a stabilizer circuits, which use only Clifford gates (Hadamard, CNOT, Phase), and is known to be simulated efficiently on classical computers via the Gottesman–Knill theorem [1]. Moreover, a key systems gap is revealed in [5, 9]: existing approaches have not yet facilitated the full power of RDBMS to handle simulation workloads such as query optimization, indexing, and emphasize that closing this gap is important for an end-to-end database-driven simulation pipeline. Altogether, these results motivate a database-centric benchmark: to move beyond isolated case studies, the community needs a systematic way to generate *diverse* simulation workloads and study how SQL representations behave under query optimization and execution. Altogether, these results point to a benchmark gap.

Why this is different from existing tensor workloads in RDBMS.

When a quantum circuit is compiled into SQL, the underlying computation *can* be expressed as tensor algebra: applying gates corresponds to multiplying and contracting tensors (e.g., updating a state vector by small operators). At a computational level, circuit simulation in RDBMS is a tensor workload: vector-matrix multiplication, matrix-matrix multiplication, and tensor contractions. One of the most frequent operations is matrix multiplication. Using SQL, matrix multiplication can be implemented as a join-and-aggregate operation: represent the matrices in long form as $A(i, k, a_{ik})$ and $B(k, j, b_{kj})$, compute $W = AB$ by joining rows of A and B on the shared inner index k , multiplying matching values, and summing over k :

$$w_{ij} = \sum_k a_{ik} b_{kj},$$

which corresponds to:

```
SELECT A.i, B.j, SUM(A.val * B.val) AS w_ij
FROM A JOIN B ON A.k = B.k
GROUP BY A.i, B.j;
```

This style of expressing tensor computation in SQL has been studied in the database community [6, 13, 18, 21, 29], including recent work on efficient Einstein summation in SQL [5] and relational abstractions for tensor and linear-algebra workloads in ML [7, 17, 24, 27, 28].

However, the simulation workload induced by quantum circuits differs from mainstream ML workloads in several fundamental ways. In transformer models, the core computation is dominated by dense linear algebra over matrices with dimensions in the thousands (e.g., attention uses dense $QK^\top$ and QV products) [26]. In contrast, a quantum circuit acts on a state vector of dimension 2^N : for qubit count $N = 50$, the state has $2^{50} \approx 1.13 \times 10^{15}$ amplitudes, which already requires about 16 PiB just to store a dense complex state vector. At the same time, each instruction applies only a tiny

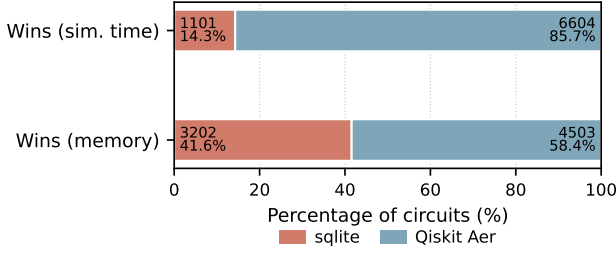


Figure 1: Across 7,705 circuits, share of cases where SQLite (executing the emitted SQL workload) or Qiskit Aer (best of its built-in simulation methods) achieves the lowest runtime and the lowest peak memory. Detailed experimental settings are in Section 6.

operator (typically 2×2 or 4×4 matrices), but it must be applied across this exponentially large index space. The structure of a quantum circuit determines a sequence of tensor contractions whose intermediate sparsity and entanglement can change abruptly across the gate sequence.

This mismatch creates database-specific difficulties: (i) intermediate results may be *enormous* unless the representation stays sparse or factored; (ii) whether sparsity survives depends on the *gate sequence*, and entangling operations can rapidly densify the state and cause abrupt blow-ups; and (iii) the computation is a long, dependency-constrained sequence of small updates rather than a few large tensor computations, so “optimize one big join” is not the right mental model. Consequently, optimizing SQL for quantum circuit simulation is not a direct reuse of existing in-database tensor/ML techniques: the bottlenecks, cardinality growth, and the importance of circuit structure (sparsity patterns, interaction graphs, composition rules) are qualitatively different. A benchmark that systematically varies these properties is therefore essential for a principled optimizer and execution-engine research in this space.

Gaps of existing benchmarks. Existing quantum benchmarks such as SupermarQ [25], MQT Bench [23], and QASM Bench [15] provide valuable circuit suites for the quantum community, but they do not directly target database execution. In particular, they generate circuits as quantum-circuit descriptions (e.g., QASM/Qiskit circuits) meant to be run by quantum toolchains, not as relational workloads. As a result, they are not designed to evaluate how an RDBMS behaves as a simulation engine, e.g., the impact of query optimization, physical design (indexes/partitioning), and execution strategies on circuit-simulation workloads.

Our approach. We present InferQ, a database-oriented benchmark for quantum circuit simulation. Given a small user configuration (Table 2), InferQ generates *compositional* circuits by assembling subcircuits from a set of quantum templates. Each circuit is accompanied by an *RDBMS-ready SQL workload* that expresses the simulation task as relational operators. Moreover, to support database-centric analysis, InferQ also extracts four groups of feature: *static* and *graph* features from the circuit structure, *SQL* features from the emitted query workload, and *dynamic* features from the simulation.

Contributions. We make the following contributions:

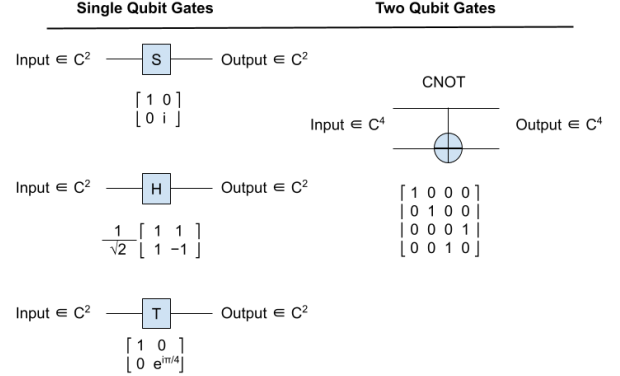


Figure 2: Quantum gates represented as matrices.

- We propose InferQ, a benchmark that produces quantum circuit simulation workloads in *relational form* (SQL), enabling direct evaluation inside RDBMS engines.
- We design a template-based generator that *scales* circuit size while preserving realistic structure via compositional circuit generation (Section 3).
- From the generated circuits, we extract *static*, *graph*, *SQL*, and *dynamic* features per circuit, enabling systematic workload characterization and correlation with database performance (Section 4).
- Using workloads generated by InferQ, we quantify when SQL-driven simulation is competitive on *general*, *compositional* circuits. InferQ features enable accurate, lightweight machine learning based selectors for deciding when to use an RDBMS engine (Section 6).

2 Background

2.1 Quantum Bits, Gates, and Circuits

A classical bit stores either 0 or 1. In contrast, a *quantum bit* (qubit) is a unit vector in a two-dimensional complex vector space, typically described using the computational basis of $|0\rangle$ and $|1\rangle$:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

An arbitrary single-qubit state can be written as a superposition of these basis states:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1,$$

where α and β are complex amplitudes. If we perform *measurement* on the quantum state $|\psi\rangle$, it collapses to a classical bit of 0 or 1. That is, a measurement of $|\psi\rangle$ against the computational basis yields outcome 0 with probability $|\alpha|^2$ and outcome 1 with probability $|\beta|^2$.

An n -qubit state can be expressed as a superposition over all computational basis states indexed by bitstrings:

$$|\Psi\rangle = \sum_{x \in \{0,1\}^n} c_x |x\rangle, \quad \sum_{x \in \{0,1\}^n} |c_x|^2 = 1.$$

Fix an ordering of the computational basis states (e.g., lexicographic order). The *state vector* representation of $|\Psi\rangle$ is the column vector

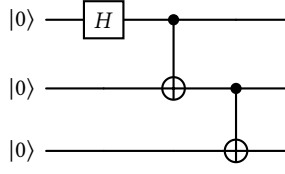


Figure 3: GHZ state preparation circuit: it takes the all-zero state $|000\rangle$ as input and prepares a maximally entangled state.

of its amplitudes in that order: $\begin{bmatrix} c_{0\dots0} \\ c_{0\dots1} \\ \vdots \\ c_{1\dots1} \end{bmatrix}$. For example, when $n = 2$, any two-qubit state $|\Psi\rangle$ can be represented as a linear combination of $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$, and $|c_{00}|^2 + |c_{01}|^2 + |c_{10}|^2 + |c_{11}|^2 = 1$. The state vector representation of $|\Psi\rangle$ is $\begin{bmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{bmatrix}$.

Quantum computation follows the circuit model, which is analogous to classical logic circuits. Computation is performed by applying quantum gates (unitary operators) acting on one or more qubits to transform the quantum state. A matrix U is *unitary* if $U^\dagger U = U U^\dagger = I$. As shown in Figure 2, a quantum gate can be represented by a unitary matrix, and with $\text{Input} \in \mathbb{C}^n$ it indicates that the gate acts on an n -dimensional state vector of complex amplitudes (where $\mathbb{C}$ denotes the complex numbers), e.g., one qubit corresponds to $\mathbb{C}^2$ and two qubits to $\mathbb{C}^4$. A gate set is called universal if it can approximate any unitary operator to arbitrary precision [20]. A widely used universal set is the Clifford gates (including the Pauli gates (X , Y , Z), the Hadamard gate H , the phase gate S , the controlled-NOT (CNOT) gate), and the T gate. Figure 2 illustrates several of these gates and their matrix representations. A *quantum circuit* is a sequence of quantum gates, and the overall computation corresponds to the composition of their unitary matrices.

Circuit depth is the number of sequential layers of quantum gates in a circuit, representing how many time steps are required to execute it when gates that act on different qubits can be applied in parallel. For example, in Figure 3, the circuit depth is 3. We can consider quantum circuits as higher-level operators composed of subcircuits. By abstracting away from hardware and physical implementation details, we can treat such structures modularly. Take the GHZ state preparation circuit in Figure 3 for instance. It can serve as a state preparation subcircuit for larger algorithms, i.e., it prepares an input state for other algorithms.

2.2 Limitations of Existing Benchmarks

Several quantum circuit benchmarks are widely used in the quantum computing community, including SupermarQ [25], MQT Bench [23], and QASM Bench [15]. While these suites are valuable for evaluating quantum circuits, they expose limited support for database-driven and data-centric studies of circuit simulation.

Gap 1: No database-native workloads. Existing benchmarks primarily distribute circuits as OpenQASM¹ and Qiskit QPY² files. These formats are well-suited for quantum toolchains, but they are

poorly aligned with database research: they do not expose simulation workloads in a form that can be directly executed, inspected, and optimized by an RDBMS. As a result, studying circuit simulation inside an RDBMS typically requires substantial manual translation and non-trivial quantum expertise. We address this gap by having InferQ generate circuits together with an RDBMS-ready SQL workload (Section 3).

Gap 2: Lack of structured compositional circuits. Real quantum programs are typically assembled as pipelines of subroutines such as state preparation, algorithmic cores, estimation procedures, and measurement. For example, the Harrow–Hassidim–Lloyd (HHL) algorithm [10] combines state preparation, Hamiltonian simulation, quantum phase estimation, and measurement. However, existing benchmarks largely treat circuits as independent, fixed templates for standalone algorithms. They do not systematically compose building blocks into larger circuits, nor do they model dependencies between subcircuits. In practice, the choice of one subroutine often constrains what can follow (e.g., Hamiltonian simulation followed by phase estimation). We address this gap in Section 3.2 by modeling conditional, history-dependent template composition as a Markov transition process.

Gap 3: Limited scalability for data-centric analysis. Because many existing benchmarks focus on fixed algorithm templates, their scalability is inherently limited: the number of instances grows slowly, and structural diversity is constrained. This limits their usefulness for data-centric workflows that require large corpora, such as applying machine learning models, cost modeling, and feature-driven analysis. We address this gap with scalable compositional generation and validate scalability experimentally in Section 6.5. We also release a large dataset of 202975 circuits to support large-scale quantum circuit simulation studies.

Gap 4: Limited circuit-as-graph and SQL-level characterization. Quantum circuits are inherently graph-structured: qubits correspond to vertices and multi-qubit gates induce interactions as edges [19]. This structure is central to simulation cost and to the shape of the induced SQL workload. Nevertheless, existing benchmarks expose few graph-derived properties and provide limited support for analyzing structural diversity across circuits. We address this gap by explicitly extracting graph features (Section 4.2) and SQL features for the query representation (Section 4.3).

Summary. Overall, existing benchmarks fall short of supporting database-driven and data-centric research on quantum circuit simulation. Addressing these gaps requires a benchmark that produces SQL workloads, supports conditional composition of subcircuits, scales to large and diverse circuits, and exposes graph- and query-level structure for systematic analysis.

3 InferQ Benchmark

InferQ has two components. First, it generates quantum circuits via templates. Second, it extracts a structured set of circuit features that support database-oriented analysis of RDBMS-backed simulation. This section presents the InferQ benchmark pipeline (Figure 4). We describe the features in Section 4.

¹<https://openqasm.com/>

²<https://quantum.cloud.ibm.com/docs/en/api/qiskit/qpy>

Table 1: Representative circuit templates and example compositions.

Current template	Full circuit template name	Next possible templates	Description	Example compositions
START	(start symbol)	StatePrep , Oracle/A , Variational	Common entry points (prepare input/eigenstate/ansatz), enabling realistic multi-block pipelines.	START → StatePrep START → Oracle/A START → Variational
StatePrep	State Preparation	HamSim , ModularExp , GroverIter , Variational	Prepares $ \psi\rangle$ / $ b\rangle$ / initial superpositions before an algorithmic core; supports StatePrep → HamSim → QPE and StatePrep → famVariational.	StatePrep → HamSim → QPE StatePrep → Variational
Variational Oracle/A	Oracle / Algorithm-A Wrapper	GroverIter , Variational , Measure	Encodes a problem-dependent subroutine A (e.g., oracle/feature map/encoding) that is naturally followed by Grover-style amplification; supports $A \rightarrow$ GroverIter → AmpEst.	A → GroverIter → AmpEst
HamSim	Hamiltonian Simulation	QPE , Variational , Random	Controlled time-evolution unitaries are typically “read out” via QPE (incl. IQFT decoding) in eigenvalue/spectral workflows; supports StatePrep → HamSim → QPE.	HamSim → QPE
ModularExp	Modular Exponentiation	QPE	Order-finding / Shor-style structure: modular arithmetic unitaries followed by phase estimation (incl. IQFT) to extract phases/periods; supports ModularExp → QPE.	ModularExp → QPE
GroverIter	Grover Iteration (Amplitude Amplification)	GroverIter (repeat), AmpEst , Measure	Amplification layers typically repeat and then transition to an estimation wrapper or terminate with measurement; supports $A \rightarrow$ GroverIter → AmpEst.	GroverIter → GroverIter GroverIter → AmpEst GroverIter → Measure
QPE	Quantum Phase Estimation	Measure , Variational , Random	Phase readout/decoding stage (typically includes an IQFT); often followed by measurement or optional continuations.	QPE → Measure QPE → Variational QPE → Random
AmpEst	Amplitude Estimation	Measure , Random	Estimation stage that typically ends in measurement outputs or proceeds to auxiliary stress-test blocks.	AmpEst → Measure AmpEst → Random
Variational	Variational Circuit Layer / Ansatz	Variational (repeat), Measure , Random	Layered ansatz patterns; measurement used for objective estimation; can be interleaved for hybrid workflows.	Variational → Variational Variational → Measure Variational → Random
Random	Random Circuit Block	Random , Measure	Diversity / stress-test tail; may extend circuit length or terminate by measurement.	Random → Random Random → Measure
Measure	Measurement	—	Final step that yields classical outcomes and ends the whole circuit generation process.	Measure

Design goal. Database benchmarks such as TPC-H³, TPC-DS⁴, and the Join Order Benchmark (JOB) [14] generate query workloads by instantiating a fixed set of *query templates*. Similarly, InferQ generates a circuit workload from a fixed set of *circuit templates*. Each circuit template generates a *subcircuit*, and an output circuit is the *ordered* composition of the generated subcircuits.

3.1 Overview

InferQ targets the compositional structure found in existing quantum circuits, where a single circuit is assembled from multiple building blocks (Table 1). This differs from template-only benchmarks that generate isolated, standalone circuits. InferQ selects an ordered sequence of *circuit templates* and instantiate each template

into a *subcircuit*. The resulting subcircuits are then composed into one benchmark circuit.

Concretely, InferQ generates one circuit instance as follows:

- (1) **Select templates.** Then it sequentially samples a template sequence using the history-dependent distribution.
- (2) **Sample Hyper-Parameters.** InferQ first samples user-configured local hyper-parameters (e.g., qubit count and depth).
- (3) **Generate subcircuits.** For each selected template, InferQ samples template parameters and generates the corresponding subcircuit.
- (4) **Output circuit.** The circuit generation process stops when a stopping rule triggers, then InferQ composes all subcircuits into a single circuit.

This design produces circuits that satisfy user constraints while supporting diverse compositions and realistic template transitions.

³<https://www.tpc.org/tpch/>

⁴<https://www.tpc.org/tpcds/>

3.2 Select Templates

In InferQ, we maintain a set of circuit templates $F = \{f_1, \dots, f_M\}$. Table 1 lists representative templates, such as `StatePrep`; the full template set is documented online.⁵ Given a selected template $f \in F$, InferQ instantiates a *subcircuit* by sampling concrete, circuit-specific parameter values. For example, from the `StatePrep` template we can generate a GHZ state preparation subcircuit with 10 qubits.

Addressing Gap 2. Database benchmarks typically instantiate each query template independently, yielding a workload that is a set (or multiset) of queries without sequential dependencies among template choices. In contrast, as discussed in Gap 2, the circuits targeted by InferQ are *ordered* compositions of subcircuits, and the choice of the next subcircuit is often constrained by (and semantically coupled to) the current one. To capture such dependencies, we model template selection as a discrete-time Markov process.

Design element 1: Explicit synergy rules. InferQ provides a set of user-configurable *synergy rules* that encode statistical dependencies between templates and bias generation toward semantically coherent pipelines, e.g., `StatePrep`→`HamSim`→`QPE`, `ModularExp`→`QPE`, and `A`→`GroverIter`→`AmpEst` (Table 1). Intuitively, these rules specify how earlier template choices should increase or decrease the probability of selecting specific templates later, making the sampling distribution explicitly history-dependent.

To explain our approach for *conditional composition*, we first introduce the following definitions and notation.

Definitions. Let $\mathcal{S}$ denote the space of subcircuits, and let T be the number of templates (equivalently, subcircuits) generated for one circuit instance. Let B denote the *context space*, i.e., the current circuit generation status (e.g., remaining allowed templates, and synergy rules in Table 1). A *circuit template* $f \in F$ is a parameterized generator

$$f : \Theta_f \times B \rightarrow \mathcal{S}.$$

Let $\text{feas}(b, f) \in \{0, 1\}$ be a hard feasibility predicate for choosing template $f \in F$ under context $b \in B$.

InferQ generates an ordered template sequence $f_1, f_2, \dots, f_T$. At step t , after selecting f_t and sampling parameters $\theta_t \in \Theta_{f_t}$, InferQ instantiates the corresponding subcircuit as

$$s_t = f_t(\theta_t, b_{t-1}),$$

where $b_{t-1} \in B$ is the current context.

At each step $t \in \{1, \dots, T\}$, InferQ maintains a categorical distribution p_t over the template set F . We write $p_t(f)$ for the probability assigned to template $f \in F$, so

$$\sum_{f \in F} p_t(f) = 1.$$

A larger value of $p_t(f)$ means that template f is more likely to be selected at step t .

Design element 2: Conditional composition as a Markov transition model. At step t , InferQ samples the next template from a categorical distribution over F . Given the current context b_{t-1} , it proceeds in two steps:

(1) Sample a template. First apply feasibility masking and renormalize:

$$\bar{p}_t(f) = \frac{p_t(f) \text{feas}(b_{t-1}, f)}{\sum_{g \in F} p_t(g) \text{feas}(b_{t-1}, g)}, \quad f \in F,$$

then sample $f_t \sim \bar{p}_t$. Next sample $\theta_t \in \Theta_{f_t}$, instantiate $s_t = f_t(\theta_t, b_{t-1})$, append s_t , and update the context:

$$b_t = \text{Upd}(b_{t-1}, f_t, \theta_t, s_t).$$

(2) Reweight to obtain the next distribution. Let $\mathcal{R}$ be the set of synergy rules, where each rule is a triple (T, U, β) with $T \subseteq F$ (trigger), $U \subseteq F$ (target), and $\beta > 0$. After selecting f_t , define the reweighted (unnormalized) probabilities

$$\hat{p}_{t+1}(f) = \bar{p}_t(f) \cdot \prod_{(T, U, \beta) \in \mathcal{R}: f_t \in T, f \in U} \beta, \quad f \in F,$$

and then apply feasibility under the updated context b_t and renormalize:

$$p_{t+1}(f) = \frac{\hat{p}_{t+1}(f) \text{feas}(b_t, f)}{\sum_{g \in F} \hat{p}_{t+1}(g) \text{feas}(b_t, g)}, \quad f \in F.$$

This mechanism induces a sequence-dependent distribution while preserving diversity through stochastic sampling, enabling semantically coherent compositions of subcircuits.

Example 3.1 (Single-step reweighting under a synergy rule). Assume the previously selected template is `StatePrep`. By Table 1, the feasible successors are $\{\text{HamSim}, \text{ModularExp}, \text{GroverIter}, \text{Variational}\}$, with a uniform distribution $p_t(\cdot) = \frac{1}{4}$. Let $\mathcal{R}$ include $(T, U, \beta) = (\{\text{StatePrep}\}, \{\text{HamSim}\}, 2)$. Then

$$\begin{aligned} \hat{p}_{t+1}(\text{HamSim}) &= \frac{1}{2}, & \hat{p}_{t+1}(\text{ModularExp}) &= \frac{1}{4}, \\ \hat{p}_{t+1}(\text{GroverIter}) &= \frac{1}{4}, & \hat{p}_{t+1}(\text{Variational}) &= \frac{1}{4}, \end{aligned}$$

and after normalization,

$$\begin{aligned} \hat{p}_{t+1}(\text{HamSim}) &= \frac{2}{5}, & \hat{p}_{t+1}(\text{ModularExp}) &= \frac{1}{5}, \\ \hat{p}_{t+1}(\text{GroverIter}) &= \frac{1}{5}, & \hat{p}_{t+1}(\text{Variational}) &= \frac{1}{5}, \end{aligned}$$

3.3 Sample Hyper-Parameters

Circuit generation is constrained by the global hyper-parameters in Table 2, which are configured by users. InferQ requires users to provide ranges for key parameters, such as qubit count, depth, repetitions count for repeatable subroutines, and number of qubits used during evaluation. A concrete value is drawn uniformly at random from each range. If a user prefers a fixed value, they set the range endpoints equal (e.g., `min_qubits=max_qubits=10`). This range-based interface provides more flexibility for users to construct workloads.

3.4 Generate Subcircuits

Each selected circuit template f is instantiated by sampling parameters from its parameter domain Θ_f . In InferQ, we write one sampled parameter tuple as

$$\theta = (n_q, d, k, r, \xi) \in \Theta_f,$$

where n_q is the qubit count, d is the depth allocated to this subcircuit, k is the evaluation-qubit, r is the repetition count for repeatable template sections, and ξ is a tuple of template-specific configuration variables.

⁵<https://anonymous.4open.science/r/InferQ-ADEF/Readme.md>

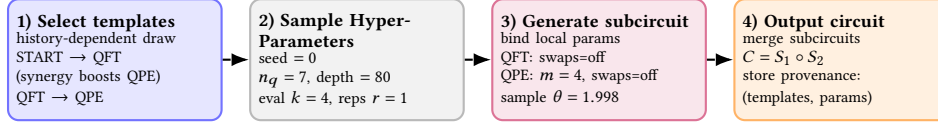


Figure 4: InferQ benchmark’s quantum circuit generation pipeline, and running example in Section 3.6.

Table 2: Hyper-parameters.

Parameter	Type	Description
min_qubits	int	Minimum number of qubits per generated circuit.
max_qubits	int	Maximum number of qubits per generated circuit.
min_depth	int	Minimum target depth of a generated quantum circuit.
max_depth	int	Maximum target depth of a generated quantum circuit.
min_reps	int	Minimum number of repetitions for repeatable subroutines (e.g., Grover iterations or variational layers).
max_reps	int	Maximum number of repetitions for repeatable subroutines.
min_eval_qubits	int	Minimum number of qubits used during evaluation.
max_eval_qubits	int	Maximum number of qubits used during evaluation.
measure	bool	Whether measurements are appended to the circuit.
seed	int	Global random seed to ensure full reproducibility of circuit generation.

The domain Θ_f is derived from the hyper-parameters in Table 2. For example, the per-subcircuit depth d is sampled uniformly from the defined range between `min_depth` and `max_depth`.

The template-specific configuration variables ξ captures local design choices supported by f . Typical examples include whether QFT includes final swaps, the choice of feature map and ansatz in variational templates, the number of layers in QAOA-style templates, or the number of steps and coin operators in quantum-walk templates.

Structural vs. numerical parameters. In InferQ, template instantiation is staged. InferQ first fixes the *structure* of the subcircuit (e.g., qubit assignment, gate topology, repetition pattern, and depth contribution) from (n_q, d, k, r, ξ) . If the template contains parametrized gates, InferQ then samples the *numerical* values (e.g., rotation angles) from template-specific distributions. This separation lets InferQ explore structural diversity independently from numerical variability.

3.5 Output Circuit

Generation stops when a maximum number of templates is reached or when a probabilistic stopping condition triggers. This prevents excessively long circuits while keeping circuit length variable across instances. After termination, the instantiated subcircuits $S_1, \dots, S_T$ are composed in order to produce the final benchmark circuit:

$$C = S_1 \circ S_2 \circ \dots \circ S_T.$$

Alongside C , InferQ outputs a provenance record that stores the template sequence $(f_1, \dots, f_r)$, sampled parameters (including repetitions and ξ), and global resource statistics (e.g., depth and gate counts). This record makes our generated circuit instances easy to inspect, query, and reproduce.

SQL query generation. Once a circuit is generated, we construct an equivalent SQL query using the implementation in [9]. We extended their package by making it a dependency to InferQ. The package uses the einsum summation (einsum) representation of a

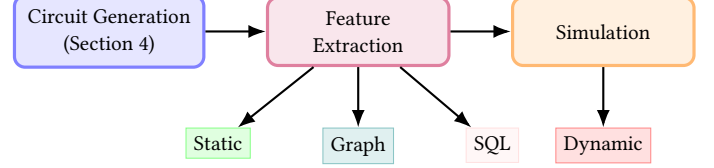


Figure 5: InferQ feature extraction workflow. Static, graph-based, and SQL-derived features are extracted prior to simulation, whereas dynamic features can only be obtained after the simulation completes.

quantum circuit with tensor algebra. It then converts tensor contractions into relational joins as proposed by Blacher et al. [5].

Reproducibility guarantee. To make sure all sampling in InferQ is deterministic, we use seeded pseudo-random number generators⁶ and maintain a global random seed as part of the hyper-parameters in Table 2. Consequently, with the seed and the generated template sequence, both the subcircuit structure and the numerical gate parameters are exactly reproducible across runs.

3.6 Running Example

Figure 4 illustrates one end-to-end circuit instance generated by InferQ. The workflow is intentionally analogous to a database benchmark that (i) samples a scale/configuration, (ii) instantiates a small sequence of templates with parameters, and (iii) materializes both an artifact and its metadata.

Step 1 (Hyper-Parameters). Given a user seed, InferQ samples an instance configuration (n_q, d, k, r, ξ) . In this example, $n_q=7$ qubits, depth $d=80$, evaluation $k=4$, and repetition count $r=1$.

Step 2–3 (Template selection & subcircuits generation). Starting from `START`, InferQ samples a short template sequence using the history-dependent distribution. Here the sequence is $QFT \rightarrow QPE$. Each template is then instantiated by binding template-local parameters (e.g., boolean flags and small numeric values such as $\theta=1.998$). All choices are deterministic given the seed.

Step 4 (Merge & provenance). The instantiated subcircuits are composed in order to produce the final circuit artifact $C = S_1 \circ S_2$. Alongside C , InferQ stores a provenance record containing the template sequence and all sampled parameters, enabling exact replay and database-style inspection.

4 Circuit Feature Extraction

When we push quantum circuit simulation into an RDBMS, it induces a sequence of relational operators (e.g., joins and aggregations implementing tensor contractions) over large intermediate relations.

⁶See <https://docs.python.org/3/library/random#random.seed> and <https://numpy.org/doc/2.2/reference/random/generated/numpy.random.seed.html>.

Table 3: Notation used in circuit feature extraction.

Notation	Description
C	Quantum circuit (ordered list of gate instructions).
G	Number of circuit instructions (gate count).
N	Number of qubits in the circuit.
W	Circuit width (maximum simultaneously active qubits).
$G_I = (V, E)$	Qubit interaction graph induced by multi-qubit gates.
V	Vertex set of G_I (qubits; $ V = N$).
E	Edge set of G_I (qubit interactions; $ E $ is the edge count).
2^N	State-space dimension (length of a full statevector).
$\mathbf{p} = (p_1, \dots, p_{2N})$	Probability vector derived from the saved statevector.
$H(\mathbf{p})$	Shannon entropy of $\mathbf{p}$ (superposition).
$\rho(\mathbf{p})$	Sparsity / support density: fraction of entries above threshold (superposition).
$S_{vN}^{(k)}$	von Neumann entropy across qubit line k (entanglement).
ϵ	Small constant for numerical stability in entropy computation.
τ	Threshold for treating near-zero entries as zero in sparsity computation.
$\mathbb{I}(\cdot)$	Indicator function (1 if condition holds, else 0).
N_{AST}	Total number of nodes in the SQL abstract syntax tree (AST).
S	Number of SELECT statements in the query.
P	Number of predicates appearing in WHERE clauses.
J	Number of distinct relations participating in join conditions.

In this setting, performance depends on cardinality growth, join selectivity, skew, and the size of intermediate states. However, such properties are not explicit in the quantum circuit representation.

To make systematic progress on query optimization, physical design (e.g., indexing/partitioning), and *routing* (deciding when an RDBMS backend is beneficial), we need a set of circuit features that indicate what the induced relational query workload looks like. Ideally, a feature vector that approximates workload characteristics *before* executing the full simulation. Therefore, we include a feature extraction component in InferQ and aim to cover different sources of cost, including problem size (number of qubits, circuit depth), interaction structure, state evolution, and the generated SQL queries. As shown in Figure 5, InferQ models each circuit with four types of features: *static*, *graph*, *dynamic*, and *SQL* features, as listed in Table 4-7, respectively. Such a separation makes InferQ **extensible**: new features can be added easily.

Notably, as shown in Figure 5, feature extraction is an optional post-processing step: InferQ can generate circuits without extracting features, and computes them only when needed for analysis.

4.1 Static Features

Static features describe basic properties of a quantum circuit. These features are crucial for constructing heuristics for efficient classical simulation. For example, dense state-vector representations are only feasible for circuits with small width and a small number of qubits, as memory requirements grow exponentially with these parameters. For an RDBMS-backed simulator, gate counts and depth largely determine the number and types of relational operators generated.

As shown in Table 4, static features can be computed efficiently. We treat a circuit as an ordered list of instructions, where each instruction applies a gate to one or more qubits. This view allows

Table 4: Time and space complexities for static features.

Feature	Time	Space	Description
num_qubits	$O(1)$	$O(1)$	Total number of qubits
width	$O(G)$	$O(W)$	Maximum simultaneous active qubits
depth	$O(G)$	$O(W)$	Longest dependency chain
circuit_size	$O(1)$	$O(1)$	Total gate count
pauli_gate_count	$O(G)$	$O(1)$	Number of Pauli gates (X, Y, Z)
two_qubit_gate_count	$O(G)$	$O(1)$	Number of two-qubit gates
two_qubit_gate_percentage	$O(G)$	$O(1)$	Ratio of two-qubit gates
locality_ratio	$O(G)$	$O(1)$	Local vs. non-local gate ratio
idling_score	$O(G)$	$O(W)$	Idle qubit utilization
density_score	$O(G)$	$O(G)$	Gate density feature [4]
gate_counts	$O(G)$	$O(G)$	Per-gate-type histogram

Table 5: Time and space complexities for graph features.

Feature	Time	Space	Description
edge_count	$O(1)$	$O(1)$	Number of interaction edges
max_degree	$O(N)$	$O(1)$	Maximum node degree
min_cut	$O(NE)$	$O(V + E)$	Minimum cut
diameter	$O(N^3)$	$O(N^2)$	Graph diameter
radius	$O(N^3)$	$O(N^2)$	Minimum eccentricity
average_degree	$O(N)$	$O(1)$	Mean node degree
average_clustering_coefficient	$O(N^3)$	$O(1)$	Local clustering tendency
average_shortest_path_length	$O(N^3)$	$O(N^2)$	Mean shortest path length
central_point_of_dominance	$O(NE)$	$O(N)$	Graph centralization measure
std_dev_adjacency_matrix	$O(N^2)$	$O(1)$	Adjacency matrix variance

us to analyze the complexity of computing static features. Let G be the number of gates (instructions), N the number of qubits, and W the circuit width (maximum simultaneously active qubits). We assume $W \leq N$ and typically $G \gg N$. We summarize the notation in Table 3. All static features in Table 4 are computable in at most linear time in G with modest memory overhead, making them inexpensive yet informative metadata for the InferQ benchmark.

4.2 Graph Features

Graph features are computed from the *qubit interaction graph*, where vertices are qubits and edges represent interactions induced by multi-qubit gates. This abstraction enables the computation of classical graph properties, similar to existing works on quantum circuit compilation [3, 4] and graph-based circuit characterization [11].

Graph features make circuit structure directly accessible to database analysis: they indicate what the join structure and intermediate-result growth may look like in relational tensor computation. Intuitively, circuits whose interaction graphs are highly connected (e.g., with diameter-reducing shortcuts, high node degree, and strong centralization) tend to induce more global correlations and fewer separable subproblems, which often translates into harder contractions and less opportunity for decomposition.

As summarized in Table 5, graph features can be computed in polynomial time and space with respect to qubit count N , with worst-case complexity bounded by $O(N^3)$. Let E be the number of edges in the interaction graph. Under a gate set dominated by one- and two-qubit gates, each gate introduces at most one interaction edge; hence $E = O(G)$, with worst-case space $O(N^2)$ for a weighted, undirected interaction graph. Building an adjacency-list representation takes $O(N + E)$ time and space. While some features require $O(N^3)$ time in the worst case, they remain practical for the

Table 6: Time and space complexities for extracting SQL features (excluding parsing).

Feature	Time	Space	Description
num_and_clauses	$O(N_{AST})$	$O(1)$	Counts logical AND operators during AST traversal
num_joins	$O(S \cdot P + J^2)$	$O(J^2)$	Counts explicit and implicit joins and stores join edges
num_select_columns	$O(N_{AST})$	$O(1)$	Counts column expressions in SELECT clauses
num_agg_funcs	$O(N_{AST})$	$O(1)$	Counts aggregate functions (e.g., COUNT, SUM, AVG)
num_where_clauses	$O(N_{AST})$	$O(1)$	Counts WHERE clause nodes in the AST
num_eq_predicates	$O(N_{AST})$	$O(1)$	Counts equality predicates used in filters and joins

Table 7: Time and space complexities for dynamic features.

Feature	Time	Space	Description
shannon_entropy	$O(2^N)$	$O(2^N)$	Shannon Entropy of stored statevector
von_neumann_entropy	$O(N \cdot 2^N)$	$O(N \cdot 2^N)$	Von Neumann Entropy of stored statevector
sparsity	$O(2^N)$	$O(2^N)$	Sparsity of stored statevector

qubit ranges where graph characterization is useful, and can be computed offline.

4.3 SQL Features

SQL features are extracted from the generated SQL queries, which are relevant to the performance of RDBMS engines for quantum circuit simulation. Table 6 summarizes the complexities to calculate these features from the circuit SQL query representation. To calculate the various joins and other clauses, we view the query tree as an abstract syntax tree (AST) and then do a traversal. Getting to this SQL representation of a quantum circuit is not expensive. Using existing methods [9, 16], we can generate these queries by translating the quantum circuit into einstein summation (einsum) notation representing the tensor contraction sequence.

In addition, we want to point out the extensibility of InferQ’s in supporting the addition of new features, which will facilitate a wide range of use cases and experimental analyses.

4.4 Dynamic Features

As shown in Table 7, dynamic features depend on the circuit’s *output state* and cannot be derived from circuit structure alone. They are crucial for database-backed simulation because relational approaches typically rely on *sparse* or *factorized* state representations: if the state remains sparse, relational operators can operate over small relations and avoid exponential blow-up; if entanglement grows quickly, intermediate results densify and the advantage diminishes. We therefore extract three dynamic features that capture these effects: (i) shannon entropy, (ii) von neumann entropy, and (iii) a sparsity (non-zero support) measure of the saved statevector.

Using the vector representation for the quantum state defined in Section 2, we define sparsity and entropies as follows: Let $\mathbf{p} = (p_1, \dots, p_{2^N})$ denote the probability distribution obtained from an N -qubit statevector in the computational basis. We compute the Shannon entropy, analogous to its use in classical and quantum information

$$H(\mathbf{p}) = - \sum_{i=1}^{2^N} (p_i + \epsilon) \log_2(p_i + \epsilon).$$

To quantify entanglement, we compute the von Neumann entropy across a partition defined by a cut along a qubit line k ,

$$S_{vN}^{(k)} = -\text{Tr}(\rho_{A_k} \log_2 \rho_{A_k}), \quad \rho_{A_k} = \text{Tr}_{B_k}(\rho),$$

InferQ Dataset
Browse and download quantum circuits with extracted features

Hash	Qubits	Depth	Gates	Connectivity	Density	Sparsity	Entanglement	Actions
f0cd8848	11	100	463	0.6073	0.8680	0.4410	0.6834	⚡ QPY ⚡ JSON ⚡ SQL
353f4288	10	103	461	0.4238	0.3450	0.2703	0.7130	⚡ QPY ⚡ JSON ⚡ SQL
c35850d8	9	106	143	0.2829	0.5707	0.9793	0.5064	⚡ QPY ⚡ JSON ⚡ SQL
2cf81c79	4	22	188	0.8422	0.4181	0.2442	0.8865	⚡ QPY ⚡ JSON ⚡ SQL
98988921	10	30	135	0.2920	0.5180	0.8232	0.2298	⚡ QPY ⚡ JSON ⚡ SQL
c8c3d11e	8	49	200	0.8079	0.5145	0.1724	0.7109	⚡ QPY ⚡ JSON ⚡ SQL
347b7e04	2	11	233	0.5821	0.1862	0.5687	0.7758	⚡ QPY ⚡ JSON ⚡ SQL
2ee740de	13	93	167	0.6393	0.2730	0.6982	0.7795	⚡ QPY ⚡ JSON ⚡ SQL
b94727e3	6	12	593	0.3311	0.3038	0.2000	0.4277	⚡ QPY ⚡ JSON ⚡ SQL

Figure 6: InferQ’s web-based UI for exploring dataset and circuit features.

where the bipartition corresponds to separating qubit line k from the remaining qubit lines. We define the sparsity

$$\rho(\mathbf{p}) = \frac{1}{2^N} \sum_{i=1}^{2^N} \mathbb{I}(p_i > \tau),$$

which we use synonymously with the support density, i.e., the fraction of basis states carrying non-negligible probability mass.

Here, the Shannon entropy $H(\mathbf{p})$ and sparsity $\rho(\mathbf{p})$ characterize the degree of superposition in the computational basis, with higher values indicating broader support over basis states, while the von Neumann entropy $S_{vN}^{(k)}$ measures quantum entanglement across each qubit line. We store the von Neumann entropy per qubit line to capture individual entanglement, rather than treating the full state as a combined vector, which being a pure state, has total entanglement entropy as zero. Characterizing both superposition and entanglement is essential for understanding the mechanisms underlying quantum advantage.

All dynamic features are expensive to compute and require exponential time and memory in N for exact computation due to the full pass over the length- 2^N vector. In particular, computing the von Neumann entropy across each of the N qubit lines incurs an additional factor of N on top of the full vector pass against the rest of the system, resulting in $O(N \cdot 2^N)$ time and memory. This motivates an additional use case for InferQ in Section 6.4: enabling accurate dynamic-feature estimators without requiring full simulation of the circuit.

5 Implementation and Artifacts

InferQ ships as (i) a Python framework that turns a user configuration (Table 2) into *circuits*, *features*, and an *RDBMS-ready SQL workload* for simulation, and (ii) a web-based UI for interactive dataset exploration (Fig. 6). Concretely, a database researcher can provide a small configuration file (e.g., qubit count) and run the toolkit to obtain SQL queries (plus circuit artifacts in JSON and QPY formats) that is ready to execute in an RDBMS engine. In addition to on-demand generation, InferQ releases a large dataset of 202,975

circuits online⁷. Both database researchers and quantum scientists can search, filter, and download circuits and feature records through the web UI.

The framework implements the pipeline described in Section 3 and Section 4. Each circuit is assigned a *content-based hash* computed from its structure and sampled parameters. This hash serves as the primary key and links the circuit artifact, the SQL workload, and all extracted feature records. For each circuit instance, InferQ stores: (i) the generated SQL queries (as .sql scripts) that represent the simulation workload, (ii) the circuit in a portable Qiskit serialization format (e.g., .qpy), and (iii) a JSON record containing the template provenance and extracted features.

Reproducibility. The code of our benchmark can be found online⁸. InferQ is designed to be reproducible. Given the same code revision, dependency lockfile, and global seed (Table 2), the framework deterministically reproduces the same template sequence, sampled parameters, circuit artifact, emitted SQL, and extracted features. The content-based hash and provenance record make each benchmark instance easy to reference, regenerate, and audit. We will include more implementation and reproducibility details in our technical report.

6 Evaluation

In this section, we use InferQ to quantify and explain when an RDBMS engine is a competitive simulation engine for *general, compositional* quantum circuits. First, in Section 6.2 we compare the InferQ-emitted SQL workloads on three RDBMS engines (PostgreSQL, SQLite, DuckDB) against Qiskit Aer for wall-clock runtime and peak memory. We have also trained lightweight selectors (linear and tree models) that predict when RDBMS should be chosen over Qiskit Aer. We validate the feature design via feature-importance analysis in Section 6.3 and demonstrate additional data-centric tasks enabled by InferQ in Section 6.4. Finally, we quantify the scalability of circuit generation and feature extraction in Section 6.5.

6.1 Experimental Settings

All experiments were run on a single workstation equipped with an Intel Core i9-12900KF (24 threads), 64 GB RAM, and an NVIDIA RTX 3090 GPU, running Ubuntu 20.04.6 LTS with Linux kernel 5.15.0. To ensure a fair comparison between different RDBMS engines and other quantum circuit simulators, we disabled GPU and other hardware acceleration; all simulations were executed on CPU.

We implemented our prototype in Python 3.12, with all dependencies pinned in `pyproject.toml`. Core numerical routines use NumPy 2.2.5, SciPy 1.15.3, and scikit-learn (≥ 1.8). For in-memory quantum-state simulation, we use Qiskit 2.0.1 together with Qiskit Aer 0.17.0. For database-backed simulation, we generate SQL using the `sql-einsum` code generator⁹ and build on the RDBMS simulator introduced by [9]. PostgreSQL is version 12.22 accessed via `psycopg2-binary` 2.9.10. SQLite is version 3.50.4, and DuckDB is version 1.1.3. Unless otherwise stated, we report the mean values across five repetitions for the experiments.

⁷https://anonymous.4open.science/r/InferQ-ADEF/analysis/training_data/estimator_training_data.parquet

⁸<https://anonymous.4open.science/r/InferQ-ADEF/Readme.md>

⁹https://github.com/ti2-group/sql-einsum/tree/main/generate_sql_code

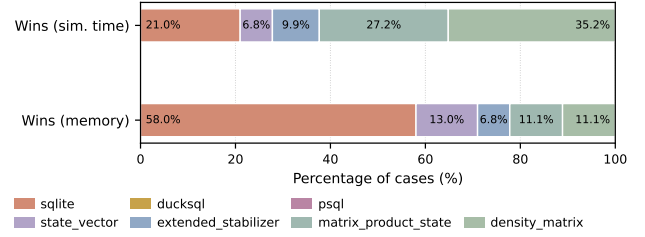


Figure 7: Execution time and memory usage comparison between RDBMSs (SQLite, DuckDB, PostgreSQL) and Qiskit Aer (state vector, stabilizer, matrix product state, density matrix) over 162 circuits generated by InferQ.

6.2 Use case 1: RDBMS as Simulation Backend

Q1: Is an RDBMS an efficient simulation engine?

To answer this question, we generated 162 circuits¹⁰ using InferQ. For each circuit, we measured wall-clock runtime and peak memory when simulating it with (i) three RDBMS engines (PostgreSQL, SQLite, and DuckDB) executing the SQL workload emitted by InferQ, and (ii) Qiskit Aer using four built-in simulation methods¹¹ (statevector, extended_stabilizer, matrix_product_state, and density_matrix).

Figure 7 reports, for each metric, the fraction of circuits for which a backend achieves the best result (minimum execution time or minimum peak memory) among all evaluated options. As expected, Qiskit Aer attains the best execution time on most circuits, since we execute the SQL workloads without database-specific tuning or query optimization. More interestingly, even in this untuned setting, SQLite achieves the lowest peak memory on 58.0% of the circuits (94/162). This suggests that RDBMS engines can be competitive simulation backends, particularly for *memory*. This result motivates further work on SQL workload optimization. Moreover, it is interesting to explore whether a learning-based routing in simulation, i.e., predicting when an RDBMS engine is likely to outperform conventional simulators, which leads to our next question.

Q2: How to decide whether to choose an RDBMS as a simulation engine?

We model this choice as a *binary selection* problem: given a circuit, predict whether an RDBMS engine achieves lower runtime and/or lower peak memory than a conventional simulator. Prior database-oriented studies of SQL-based simulation report strong results on a small set of highly structured circuits (e.g., sparse state-preparation workloads) [5, 9]. Using InferQ, we move beyond these constrained cases and quantify when an RDBMS is beneficial over a substantially broader set of general circuits. We have created a training dataset of 7,705 circuits¹² and train five standard models as selectors: logistic regression, linear SVM, Decision Tree, random forest (RF), and XGBoost. For each circuit, we have logged runtime

¹⁰https://anonymous.4open.science/r/InferQ-ADEF/analysis/training_data/rdbms_analysis_methods_training_data.parquet

¹¹Qiskit Aer Simulation Methods: https://qiskit.github.io/qiskit-aer/tutorials/1_aersimulator.html#Simulation-Method-Option

¹²Training data (Parquet) available at https://anonymous.4open.science/r/InferQ-ADEF/analysis/training_data/rdbms_training_data.parquet. The results reported in Figure 1 also used this dataset.

Table 8: Routing performance on the 20% held-out test set. Models are evaluated using accuracy and normalized confusion-matrix entries. The columns represent true label $\rightarrow$ prediction label. Best linear and tree-based models are bolded.

(a) Time-optimal routing					
Model	Accuracy	rdbms $\rightarrow$ rdbms	qiskit $\rightarrow$ qiskit	rdbms $\rightarrow$ qiskit	qiskit $\rightarrow$ rdbms
Logistic Regression	0.887	0.94	0.88	0.06	0.12
Linear SVM	0.908	0.93	0.90	0.07	0.10
Decision Tree	0.940	0.78	0.97	0.22	0.03
Random Forest	0.956	0.81	0.98	0.19	0.02
XGBoost	0.953	0.88	0.96	0.12	0.04

(b) Memory-optimal routing					
Model	Accuracy	rdbms $\rightarrow$ rdbms	qiskit $\rightarrow$ qiskit	rdbms $\rightarrow$ qiskit	qiskit $\rightarrow$ rdbms
Logistic Regression	0.935	0.93	0.94	0.07	0.06
Linear SVM	0.940	0.93	0.95	0.07	0.05
Decision Tree	0.959	0.94	0.97	0.06	0.03
Random Forest	0.973	0.95	0.99	0.05	0.01
XGBoost	0.976	0.96	0.99	0.04	0.01

and peak memory for Qiskit Aer and the RDBMS backends. This yields ground-truth labels (*RDBMS wins* vs. *Qiskit wins*) for both objectives.

Table 8a shows that these models achieve high accuracy using the features extracted in Section 4, despite class imbalance (since SQLite is faster only for a subset of circuits). XGBoost performs best overall (95.3% accuracy, lower off diagonal confusion than RF), consistent with its effectiveness for learned cost models [2, 8]. The best linear model, linear SVM, too achieves 90.8% accuracy with very low off diagonal confusion entries on the test set, suggesting that much of the time-selection boundary is close to linear in our feature space. Given the strong performance of these lightweight models, we did not evaluate more complex models that require substantially higher training and tuning cost.

For memory efficiency (Table 8b), performance is similarly strong: random forest and XGBoost are both best (with above 97.3% accuracy and less than 5% misclassification per class), while the linear SVM also remains highly competitive (94.0% accuracy). Overall, InferQ features in Section 4 make the RDBMS selection decision predictable with simple, practical models.

Take-away. (1) Across a wide range of *general, compositional* circuits, RDBMS engines can be competitive simulation engines, especially in peak memory, and in some cases also in runtime. *This enables follow-up work on optimizing the emitted SQL to further improve RDBMS performance.*

(2) InferQ enables data-centric routing at scale: it generates diverse circuits, labels each circuit with observed runtime and peak memory, and exposes features that allow simple models to decide when an RDBMS engine should be used. *This provides a foundation for learning-based simulator selection and optimization, trained on InferQ-generated circuits.*

6.3 Feature Validation

Next, we want to go deeper and understand:

Q3: Which properties determine whether an RDBMS engine is an efficient simulation engine?

Our goal is to validate that the four feature groups introduced in Section 4 (*static, graph, dynamic, SQL*) capture the key signals behind the routing outcome in Section 6.2. We therefore analyze feature importance for the best models: XGBoost for *runtime* and Random Forest for *peak memory*. We additionally report a Linear SVM to expose the *direction* of influence via signed coefficients.

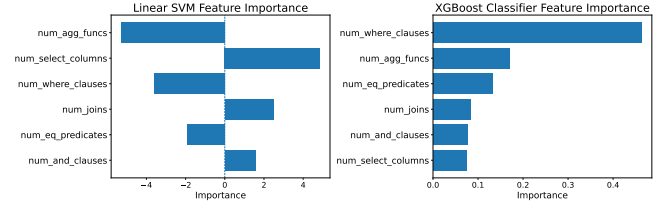


Figure 8: Importance of various SQL features for SVM and XGBoost trained for optimizing execution time.

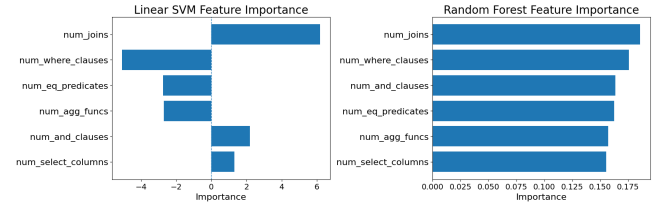


Figure 9: Importance of various SQL features for SVM and RF trained for optimizing memory.

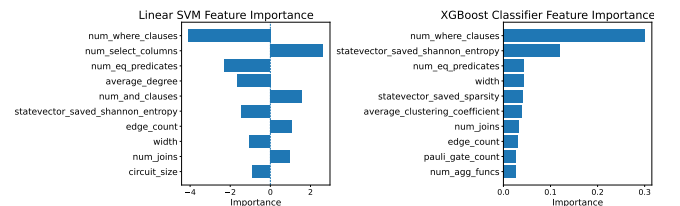


Figure 10: Importance of top 10 InferQ numeric features for SVM and XGBoost trained for optimizing execution time.

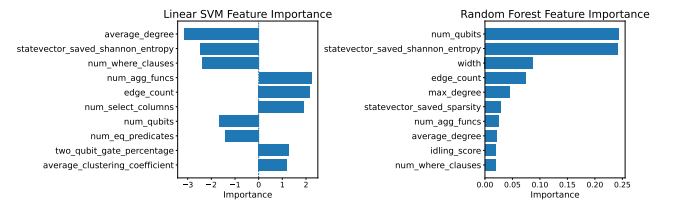


Figure 11: Importance of top 10 InferQ numeric features for SVM and Random Forest trained for optimizing memory.

Table 9: Routing performance using only SQL-derived features on the 20% held-out test set. Models are evaluated using accuracy and normalized confusion-matrix entries. The columns represent true label $\rightarrow$ prediction label. Best linear and tree-based models are bolded.

(a) Time-optimal routing					
Model	Accuracy	rdbms $\rightarrow$ rdbms	qiskit $\rightarrow$ qiskit	rdbms $\rightarrow$ qiskit	qiskit $\rightarrow$ rdbms
Logistic Regression	0.787	0.88	0.77	0.12	0.23
Linear SVM	0.803	0.88	0.79	0.12	0.21
Decision Tree	0.915	0.76	0.94	0.24	0.06
Random Forest	0.932	0.75	0.96	0.25	0.04
XGBoost	0.924	0.81	0.94	0.19	0.06

6.3.1 SQL-only features. We first restrict to SQL-derived features extracted from output SQL queries representing the generated circuits (Section 4.3, Table 6). Table 9a shows that these SQL-only features already yield strong routing accuracy, especially for time-optimal routing (XGBoost exceeds 92% accuracy on the held-out test set). This indicates that the engine choice is strongly correlated with *query shape*, without requiring circuit-specific features.

Figures 8 and 9 show that the dominant SQL signals are structural counts, in particular the number of JOINS, predicates, and aggregates. For runtime, the Linear SVM assigns negative coefficients to num_where_clauses, num_eq_predicates, and num_agg_funcs, suggesting that queries with heavier filtering and aggregation tend to favor the RDBMS backend. In contrast, num_joins and num_and_clauses shift the decision toward Qiskit Aer. For memory, num_joins is the dominant factor, while predicate and aggregation features retain negative influence, reflecting that relational filtering/aggregation can reduce intermediate sizes.

6.3.2 All InferQ features. We then include the full feature set from Section 4. For runtime (Figure 10), for both the Linear SVM and XGBoost, features associated with state growth and interaction intensity are more important, including shannon_entropy, width, and edge_count, together with the SQL-derived features. With XGBoost, in addition, graph-related features are important, such as average_clustering_coefficient, average_shortest_path_length, and average_degree, indicating that irregular interaction structure matters for runtime differences.

For peak memory (Figure 11), the Linear SVM and Random Forest agree on the main features: num_qubits and shannon_entropy dominate, with additional contributions from width, edge_count, degree statistics, and sparsity measures. The Random Forest captures non-linear thresholds in these features, separating regimes where sparse relational execution substantially reduces peak memory.

Take-away. (1) SQL-only features are already strong predictors: the SQL query workload shape (joins, predicates, aggregates) explains much of the whether RDBMS should be chosen. *This enables follow-up work on SQL-level cost modeling and query optimization tailored to quantum circuit simulation workloads.*

(2) When using the full feature set, the top features are consistent with the underlying bottlenecks: state growth indicators such as entropies, circuit scale like qubits count and circuit width, and interaction structure (graph features). *This supports follow-up work on data-centric models that learn when to use an RDBMS engine and how to optimize execution from InferQ-generated training data.*

(b) Memory-optimal routing

Model	Accuracy	rdbms $\rightarrow$ rdbms	qiskit $\rightarrow$ qiskit	rdbms $\rightarrow$ qiskit	qiskit $\rightarrow$ rdbms
Logistic Regression	0.735	0.77	0.71	0.23	0.29
Linear SVM	0.737	0.77	0.72	0.23	0.28
Decision Tree	0.836	0.81	0.85	0.19	0.15
Random Forest	0.866	0.82	0.90	0.18	0.10
XGBoost	0.842	0.84	0.85	0.16	0.15

Table 10: Sparsity and Entropy (Dynamic metrics) estimators performance for Random Forest and Linear Regression models. Reported are coefficient of determination (R^2) and root mean squared error (RMSE) comparing predictions to actual values.

Model	Sparsity Model	Entropy Model
Random Forest R^2	0.7737	0.8936
Random Forest RMSE	0.1928	1.9931
Linear Regression R^2	0.3934	0.6956
Linear Regression RMSE	0.3156	3.3714

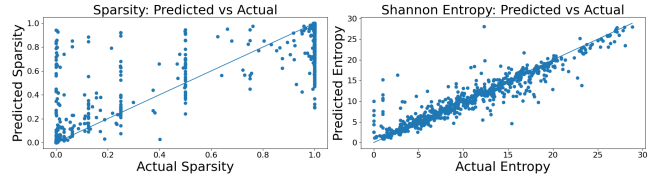


Figure 12: Prediction vs Actual value for the dynamic feature RF regression estimators for subset (1000 entries) of the test set.

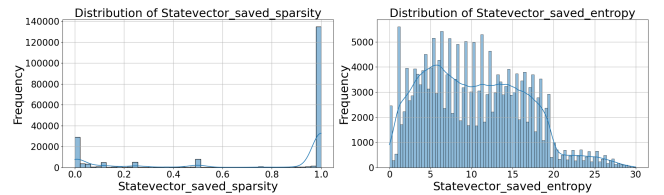


Figure 13: Distribution of dynamic features over 202k entries in InferQ with Dynamic features.

6.4 Use case 2: Predicting Entropies and Sparsity

Dynamic features from Section 4 (Shannon entropy, von Neumann entropy, and sparsity) are only available *after* simulation, since they are computed from the output state. This limits their use for larger circuits and motivates a second, data-centric use case: learning inexpensive estimators that predict these dynamic properties from static/graph/SQL features. Besides being useful for database researchers (use case 1), these metrics are also fundamental to quantum analysis.

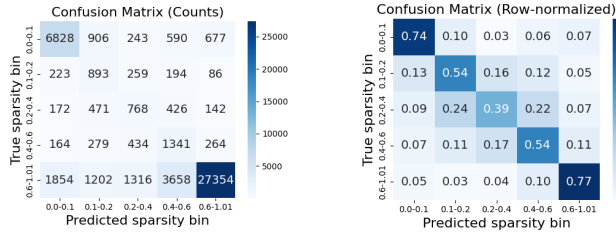


Figure 14: Confusion matrix with counts and normalisation for random forest classifier for sparsity band prediction.

Q4: Can we estimate complex quantum properties using InferQ features?

We construct estimators for sparsity and shannon_entropy using 202,975 circuits for which dynamic features are available.¹³ We train (i) a Random Forest regressor and (ii) a linear regression baseline on an 80/20 train-test split, and evaluate prediction quality using R^2 and RMSE as shown in Table 10.

As Table 10 shows, Random Forest substantially outperforms the linear model for both targets. Shannon entropy is modeled well by the feature set, while sparsity is harder to regress accurately. This is also visible in the prediction-versus-actual plots in Figure 12.

To understand why sparsity is challenging, we inspect its empirical distribution (Figure 13). Sparsity is not smoothly distributed. Instead, it concentrates in distinct bands. This suggests treating sparsity as a coarse-grained classification problem. Concretely, we bin sparsity into $[0.0, 0.1)$, $[0.1, 0.2)$, $[0.2, 0.4)$, $[0.4, 0.6)$, $[0.6, 1.01]$ and train a Random Forest classifier. This yields approximately 77% accuracy, and the normalized confusion matrix is concentrated near the diagonal (Figure 14), indicating that InferQ features capture sparsity even when exact regression is difficult.

Finally, feature-importance analysis for the sparsity-band classifier highlights pauli_gate_count as a stronger predictor than num_qubits. This suggests that (in our generated workload) sparsity is driven more by the gate mix than by state dimension alone, offering a concrete, data-derived signal that can guide both simulator design and feature engineering.

Take-away. (1) InferQ enables learning *cheap estimators* for expensive dynamic properties (e.g., entropy and sparsity) using only features that do not require simulation. *This enables follow-up work on fast, simulation-free predictors of state complexity for larger circuits.*

(2) The learned models reveal structure in the data (e.g., sparsity bands) and highlight which circuit characteristics are most predictive. *This lays a practical foundation for improving estimators (e.g., hybrid classification/regression) and for connecting circuit structure to quantum-state behavior in a data-centric manner.*

Besides this use case, we also evaluate using InferQ to select the most suitable quantum data representation (i.e., simulation method) in Qiskit Aer. This is a well-studied topic in quantum

¹³ Estimator training data (Parquet) available at https://anonymous.4open.science/r/InferQ-ADEF/analysis/training_data/estimator_training_data.parquet.

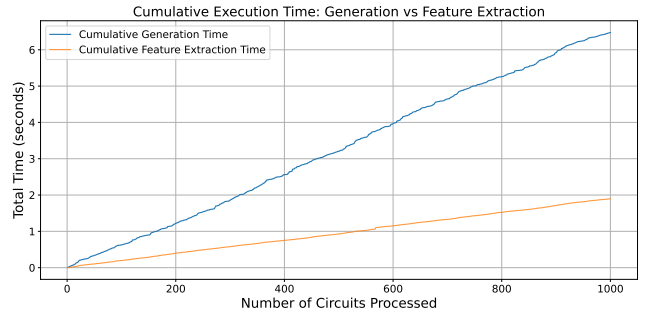


Figure 15: Cumulative execution time versus number of circuits processed. The figure shows cumulative generation time, and cumulative feature extraction time.

circuit simulation and is also posed as a data management research question in [9]. Due to space constraints, we report these results in our technical report.

6.5 Scalability

Figure 15 reports how the InferQ preprocessing cost grows as we increase the number of generated circuits. We observe near-linear scaling for both *circuit generation* and *feature extraction* (static, graph, and SQL features): generating 1,000 circuits takes only 6.47 seconds, and extracting features takes 1.9 seconds.¹⁴ This tackles Gap 3 in Section 2.2. That is, InferQ can scale to large circuit corpora: generation and feature extraction contribute negligible overhead.

Take-away. InferQ scales well in preprocessing: circuit generation and (static/graph/SQL) feature extraction grow linearly and remain inexpensive. This supports building large circuit datasets for database- and learning-based analyses.

7 Conclusion

InferQ makes SQL-based quantum circuit simulation accessible as a database benchmarking problem: it generates compositional circuits as RDBMS-ready SQL workloads, attaches features that explain relational workload difficulty, and provides both an on-demand toolkit and a large dataset for reproducible analysis. Our evaluation shows that RDBMS engines can be competitive, especially in peak memory across a broad set of general circuits, and that lightweight models trained on InferQ features can reliably predict when an RDBMS engine will be preferable. We expect InferQ to enable follow-up work in (i) query optimization and physical design for emitted simulation SQL, and (ii) data-centric routing and cost modeling for simulator selection, trained at scale on InferQ-generated circuits and features.

¹⁴ We exclude dynamic features here, as extracting them requires running the simulation, while Section 6.4 suggests that they can be predicted using static, graph, and SQL features.

Appendices

A Subcircuit generation

In Section 3 we mentioned that for each template f , we have parameter domain Θ_f . Here we explain the below parameters that included in Θ_f . From Θ_f , InferQ decides the value of the parameter through sampling. For instance, suppose a user sets `min_depth=5` and `max_depth=10`. These bounds apply to each generated *subcircuit* independently. When a template f is selected, InferQ forms the domain of its depth parameter d by intersecting the template’s intrinsic depth range with $[5, 10]$. InferQ then samples a value d from this per-subcircuit domain and instantiates a subcircuit of exactly that depth. The total depth of the final circuit is the sum of the depths of all instantiated subcircuits.

$$(n_q, d, k, r, \xi),$$

where:

- n_q is the number of qubits;
- d is a target depth;
- k is the number of qubits used for evaluation tasks;
- r is a repetition count for repeatable subroutines inside a generator;
- ξ denotes a collection of *template-specific configuration variables*.

The variable ξ captures auxiliary design choices that are local to a given circuit generator and are not shared across the global benchmark configuration. These include discrete architectural options, boolean feature flags, local layer counts, and generator-specific control parameters. All such variables are sampled by family-specific parameter generators and are constrained only by the global hyperparameters and the current circuit context. Typical examples of variables contained in ξ include whether a Quantum Fourier Transform includes final qubit-reversal swaps, the choice of feature map and ansatz type in variational circuits, the number of layers in QAOA-like subroutines, or the number of steps and coin operators in quantum walk circuits.

Once a template is selected, a structural circuit template is first instantiated using the current instance configuration and the generator’s internal settings. This defines the qubit layout, gate topology, repetition structure, and depth contribution of the subcircuit. If the generator includes parametric gates, numerical gate values are then sampled from generator-specific distributions. This staged construction separates architectural variability from numerical variability, allowing InferQ to explore both circuit structure and parameter space in a controlled and independent manner.

B Implementation

InferQ is supported by two complementary tools: (1) a *generation and feature-extraction framework* used to construct the dataset, and (2) a *web-based dataset browser* for interactive exploration and retrieval.

B.1 InferQ Generation and Feature Extraction Framework

The parameters in Table 2 are stored in Python dataclass.

The InferQ framework implements the circuit generation and metric extraction pipeline described in Section 3. It produces combinational quantum circuits and computes their static, graph, SQL, and dynamic features in a reproducible and structured manner.

Each generated circuit is assigned a *content-based hash* that uniquely identifies its structure and parameters. This hash serves as the primary key across the dataset and links the circuit to all extracted feature records.

For every circuit, InferQ stores:

- the quantum circuit in a portable serialization format compatible with Qiskit, and
- a feature record in JSON format containing all static, graph, and dynamic metrics.

Feature extraction is modular and category-based. Static features are derived directly from the circuit representation, graph features are computed from the qubit interaction graph, and dynamic features are obtained from full-state simulation. This separation allows InferQ to be extended with new feature families without recomputing or modifying existing entries.

B.2 InferQ Web Dataset Viewer

InferQ is accompanied by a web-based dataset browser inspired by MQTBench[23] that enables interactive inspection, filtering, and download of circuits and their features.

The interface presents the dataset as a table where each row corresponds to a circuit identified by its hash. Columns are grouped into the four InferQ feature categories: *static*, *graph*, *SQL*, and *dynamic*.

Users can enable or hide individual feature columns or entire feature groups, apply range-based filters on any visible numeric feature, and explore circuits based on structural, topological, or physical properties.

Each circuit entry includes download links for both the quantum circuit file and its corresponding feature JSON record, allowing users to retrieve exactly the data selected through the interface and use it directly in simulators, learning pipelines, or benchmarking workflows.

Reproducibility and Code. InferQ is fully reproducible: given the same code revision, dependency lockfile, and random seed, the framework deterministically regenerates identical circuit structures, parameters, and extracted features. Each circuit is assigned a content-based hash that uniquely identifies its structure and serves as the primary key linking the circuit artifact to all feature records.

Code and environment. InferQ is released as open-source software with separate modules for circuit generation, feature extraction, and dataset management. The repository includes a locked Python environment via `uv.lock` and a declared interpreter version. For exact reproduction, users should report the git commit hash, Python version, and lockfile version.

Cloud-agnostic storage. Circuit binaries and feature files are stored in a cloud object store. While Azure Blob Storage is used in the reference deployment, all storage operations are routed through a backend-agnostic interface, allowing any S3-compatible or local filesystem to be used. Each circuit is stored in its own directory `Circuits/{circuit_hash}`, containing a serialized `.qpy` circuit

file and a .json metadata file that records generator history and extracted features.

Deterministic circuit generation. Circuit construction follows an incremental process in which subcircuits (“generators”) are sampled and appended until a stochastic stopping condition is met. All randomness is controlled by a single global seed, which governs instance-level parameters (e.g., qubit count and depth), generator selection, generator-internal choices, and gate parameters.

Synergy-based generator sampling. Generator selection is history-dependent rather than i.i.d. After each generator is chosen, the categorical distribution over the remaining generators is reweighted using predefined *synergies* between generator families. These synergies favor complementary compositions (e.g., state preparation followed by algorithmic cores and optional variational layers), discourage long runs of the same family, and enforce feasibility constraints such as depth and qubit limits. The updated distribution is then renormalized before sampling the next generator. Because these updates are deterministic functions of the previous generator sequence and the global seed, the entire generator sequence is exactly reproducible.

Feature reproducibility. For each circuit, InferQ extracts static, graph-based, and dynamic (simulation-based) features. Feature extractors are modular and deterministic given the same simulator backend, numeric precision, and hardware configuration. For fair comparison, users should report simulator version and floating-point settings when dynamic features are used.

B.3 Qiskit Aer Data Representation Selector

Most quantum applications will be run using a simulator framework like IBM Qiskit [12] with Qiskit Aer[22] as a simulation backend. Qiskit Aer offers various simulation backends corresponding to different quantum data representations. Each performs better or worse in terms of time and memory depending on the circuit to be simulated. This leads to the research question: *what is the optimal Qiskit Aer data representation for classical simulation?*

InferQ can be used to train a selector for the Qiskit data representations for optimal simulation. In our experiment we trained various ML models to select the best representation within the classes of [“statevector”, “density_matrix”, “extended_stabilizer”, “matrix_product_state”] provided by Qiskit Aer. We recorded the execution time for 93,567 entries in InferQ on our machine from Section 5.¹⁵ For this experiments, we used the framework from Qymera [16].¹⁶

By defining the best representation to be the one taking lowest execution time, we could train various multiclass classifier models to select the best method.

Training models of complexities and evaluating them on a test set of size 20% outperformed Qiskit Aer’s own “automatic” data representation selector by a margin of up to 28%.

The accuracy in table 11 is the percentage of cases in the test set that the predicted best method matched the actual best method. Upon inspection of the features, it became clear why the Naive

Table 11: Qiskit Aer’s method selector accuracy gain for InferQ-trained models over Qiskit Aer’s built-in “automatic” selector.

Metric	Qiskit “Automatic”		Trained on InferQ				Naive Bayes
	Accuracy	XGBoost	Random Forest	Decision Tree	KNN	Logistic Regression	
Accuracy	36%	64%	64%	63%	61%	61%	55%

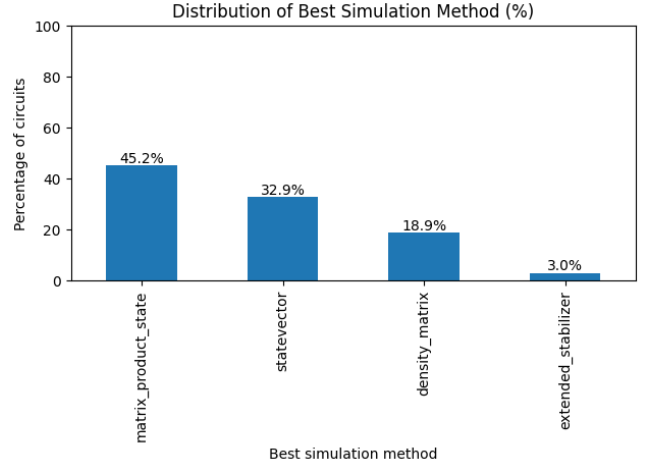


Figure 16: Bar chart displaying the imbalance between the classes being the best data representation for minimum execution time.

Bayes performs weakly. There is multi modality within feature distributions rather than Gaussian structures. In addition, the classes are highly imbalanced as shown in Figure 16.

Despite the K-Nearest Neighbour (KNN)’s relatively high accuracy, it is clear from Figure 17 that it is unable to predict the smaller classes well, as 42% of the “density_matrix” and 34% “extended_stabilizer” methods are being classified as “matrix_product_state”. The Decision Tree is much better overall for all the classes, except it is overcompensating for the minority classes and hence has low accuracy for the dominant “matrix_product_state”. Logistic regression has lowest accuracy so we are left with tree models and linear SVM.

In order to break this tie, we can look at the distribution in which the simulation methods come 2nd, 3rd and 4th place. From 12 it becomes clear that “matrix_product_state”, “density_matrix” and “statevector” dominate in 2nd position too. Hence, we can choose XG boost which performs the best for these 3 dominant classes as can be seen from confusion plots of Figure . Although the accuracy of XGBoost is 64.4%, it still outperforms Qiskit Aer’s own “automatic” method which is only able to select correctly in 35.9% of the cases. Even simpler models like logistic regression and linear SVMs are able to leverage InferQ features to perform relatively well against the current Qiskit Aer “automatic” selector.

We thus show that InferQ can be used to optimise Qiskit Aer Workloads by building a selector, showing that a complex model like XGBoost surpasses its simpler competitors in predictions amongst the highly imbalanced classes of representations. By outperforming

¹⁵Qiskit Selector training data (Parquet) available at https://anonymous.4open.science/r/InferQ-ADEF/analysis/training_data/selector_training_data.parquet

¹⁶We thank the authors for sharing their code and helping us set up the experiments.

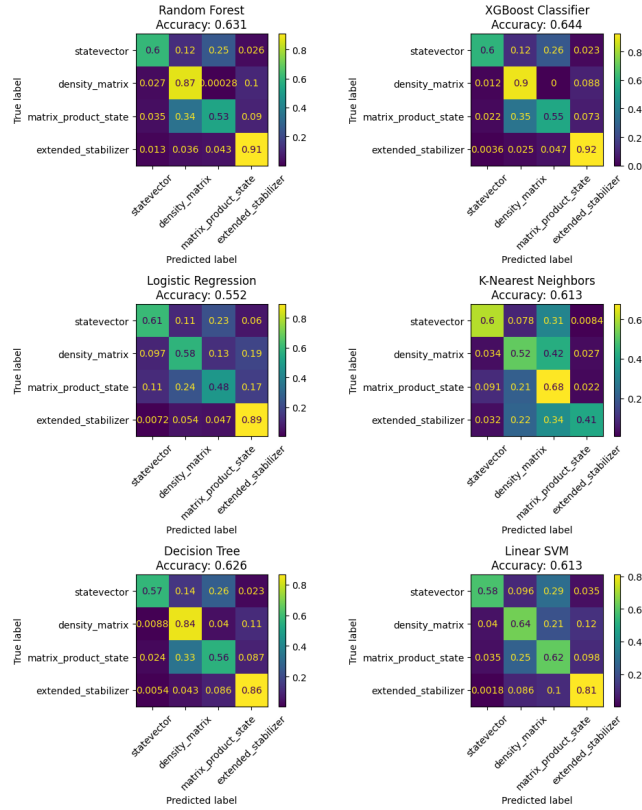


Figure 17: Confusion matrices, normalised over the classes for each model’s performance on the test set.

Table 12: Number of entries where each simulation method is ranked 1st, 2nd, 3rd, and 4th by the XGBoost-based selector, ordered by predicted execution time.

Method	Rank Position			
	1	2	3	4
statevector	30802	34743	25131	2891
density_matrix	17704	20340	7625	948
matrix_product_state	42288	35959	14537	783
extended_stabilizer	2773	2525	46274	41995

Qiskit Aer’s own “automatic” selector with all models experimented with, it validates our data-centric approach to this problem.

Moreover, the experiment showcases that data management techniques can be used to make classical simulation of quantum computation more efficient without incorporating the domain specific techniques of quantum science. It also completes a pipeline to solve the multi-class problem of which classical simulation method is optimal given a circuit incorporating the novel approach of RDBMS.

C InferQ Feature Schema

Figure 18 illustrates the schema of how InferQ stores the extracted circuit features.

References

- [1] Scott Aaronson and Daniel Gottesman. 2004. Improved simulation of stabilizer circuits. *Physical Review A* 70, 5 (Nov. 2004). doi:10.1103/physreva.70.052328
- [2] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, et al. 2019. Learning to optimize halide with tree search and random programs. *ACM Transactions on Graphics (TOG)* 38, 4 (2019), 1–12.
- [3] Medina Bandic, Carmen G. Almudever, and Sebastian Feld. 2023. Interaction graph-based characterization of quantum benchmarks for improving quantum circuit mapping techniques. *Quantum Machine Intelligence* 5, 2 (Oct. 2023). doi:10.1007/s42484-023-00124-1
- [4] Medina Bandic, Pablo le Henaff, Anabel Ovide, Pau Escofet, Sahar Ben Rached, Santiago Rodrigo, Hans van Someren, Sergi Abadal, Eduard Alarcon, Carmen G. Almudever, and Sebastian Feld. 2024. Profiling quantum circuits for their efficient execution on single- and multi-core architectures. arXiv:2407.12640 [quant-ph] <https://arxiv.org/abs/2407.12640>
- [5] Mark Blacher, Julien Klaus, Christoph Staudt, Sören Laue, Viktor Leis, and Joachim Giesen. 2023. Efficient and Portable Einstein Summation in SQL. *Proceedings of the ACM on Management of Data (PACMOD)* 1, 2, Article 121 (jun 2023), 19 pages.
- [6] Matthias Boehm, Matteo Interlandi, and Chris Jermaine. 2023. Optimizing Tensor Computations: From Applications to Compilation and Runtime Techniques. In *Companion of the 2023 International Conference on Management of Data (SIGMOD)*. 53–59.
- [7] Lingjiao Chen, Arun Kumar, Jeffrey Naughton, and Jignesh M Patel. 2017. Towards Linear Algebra over Normalized Data. *Proceedings of the VLDB Endowment* 10, 11 (2017).
- [8] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
- [9] Rihan Hai, Shih-Han Hung, Tim Coopmans, Tim Littau, and Floris Geerts. 2025. Quantum Data Management in the NISQ Era. *PVLDB* 18, 6 (2025), 1720–1729.
- [10] Aram W Harrow, Avinandan Hassidim, and Seth Lloyd. 2009. Quantum Algorithm for Linear Systems of Equations. *Physical Review Letters* 103, 15 (2009), 150502.
- [11] Javier Martín Hernández and Piet Van Mieghem. 2015. Classification of graph metrics. <https://api.semanticscholar.org/CorpusID:37136216>
- [12] IBM Quantum. 2025. Qiskit: Open-Source Quantum Computing Software. <https://www.ibm.com/quantum/qiskit>. Official IBM Quantum page for Qiskit, the world’s most popular software stack for quantum computing and algorithms research..
- [13] Mahmoud Abo Khamis, Hung Q. Ngo, Xuanlong Nguyen, Dan Olteanu, and Maximilian Schleich. 2020. Learning Models Over Relational Data Using Sparse Tensors and Functional Dependencies. *ACM Transactions on Database Systems (TODS)* 45, 2 (2020).
- [14] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.
- [15] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2022. QASM-Bench: A Low-level QASM Benchmark Suite for NISQ Evaluation and Simulation. arXiv:2005.13018 [quant-ph] <https://arxiv.org/abs/2005.13018>
- [16] Tim Littau and Rihan Hai. 2025. Qymera: Simulating Quantum Circuits using RDBMS. In *Companion of the 2025 International Conference on Management of Data (SIGMOD/PODS ’25)*. ACM, 179–182. doi:10.1145/3722212.3725126
- [17] Shangyu Luo, Dimitrije Jankov, Binhang Yuan, and Chris Jermaine. 2021. Automatic optimization of matrix implementations for distributed machine learning and linear algebra. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 1222–1234.
- [18] Nantia Makrynioti and Vasilis Vassalos. 2019. Declarative Data Analytics: A Survey. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 33, 6 (2019), 2392–2411.
- [19] Igor L. Markov and Yaoyun Shi. 2008. Simulating Quantum Computation by Contracting Tensor Networks. *SIAM J. Comput.* 38, 3 (2008), 963–981.
- [20] Michael A Nielsen and Isaac L Chuang. 2010. *Quantum Computation and Quantum Information*. Cambridge university press.
- [21] Matteo Paganelli, Paolo Sottovia, Kwanghyun Park, Matteo Interlandi, and Francesco Guerra. 2023. Pushing ML Predictions into DBMSs. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 35, 10 (2023), 10295–10308.
- [22] Qiskit Development Team. 2025. Qiskit Aer Documentation. <https://qiskit.github.io/qiskit-aer/>. Online documentation for Qiskit Aer, the high-performance

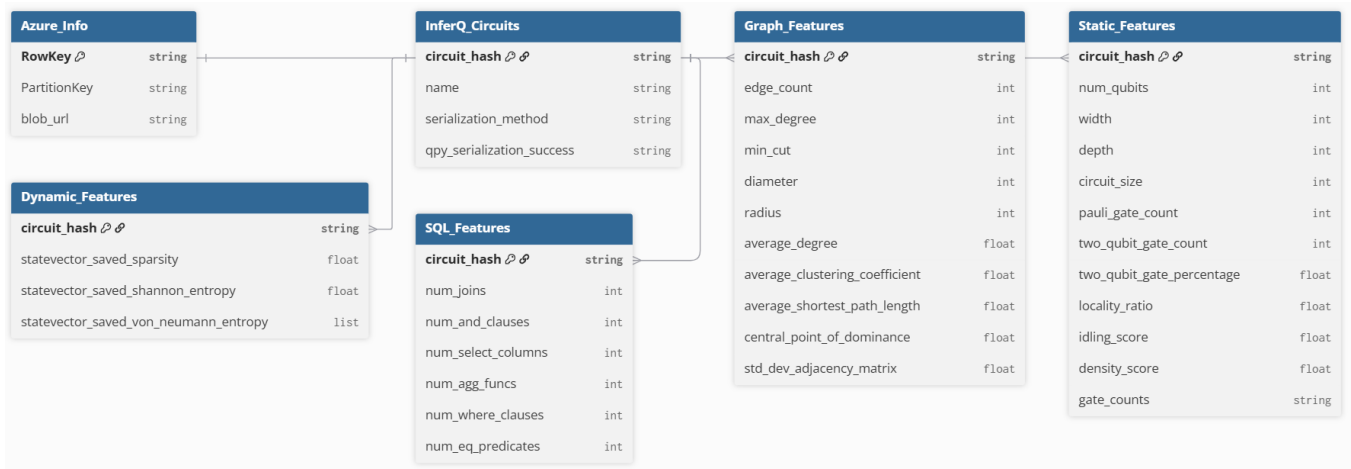


Figure 18: Schema for storing extracted circuit features. InferQ circuit data model schema. Each circuit is stored with a unique hash (primary key) and associated feature classes. InferQ entries are centered around circuits in the InferQ_Circuits table, which reference static, graph, SQL and dynamic features, as well as storage metadata in Azure DB.

- quantum circuit simulator with realistic noise models in the Qiskit ecosystem..
- [23] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing. *Quantum* 7 (July 2023), 1062. doi:10.22331/q-2023-07-20-1062
 - [24] Wenbo Sun, Qiming Guo, Wenlu Wang, and Rihan Hai. 2025. TranSQL+: Serving Large Language Models with SQL on Low-Resource Hardware. *SIGMOD* 3, 6, Article 371 (Dec. 2025), 27 pages.
 - [25] Teague Tomesh, Pranav Gokhale, Victory Omole, Gokul Subramanian Ravi, Kaitlin N. Smith, Joshua Viszlai, Xin-Chuan Wu, Nikos Hardavellas, Margaret R. Martonosi, and Frederic T. Chong. 2022. SupermarQ: A Scalable Quantum Benchmark Suite. arXiv:2202.11045 [quant-ph] <https://arxiv.org/abs/2202.11045>
 - [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
 - [27] Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: sum-product optimization via relational equality saturation for large scale linear algebra. *Proceedings of the VLDB Endowment* 13, 12 (2020), 1919–1932.
 - [28] Binhang Yuan, Dimitrije Jankov, Jia Zou, Yuxin Tang, Daniel Bourgeois, and Chris Jermaine. 2021. Tensor relational algebra for distributed machine learning system design. *Proceedings of the VLDB Endowment* 14, 8 (2021).
 - [29] Xuanhe Zhou, Chengliang Chai, Guoliang Li, and Ji Sun. 2020. Database Meets Artificial Intelligence: A Survey. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* (2020).